
Chapter 1- Introduction

Shilpa Das

Assistant Professor-II

School of Computer Engineering, KIIT DU

Reference Books:

- 1) Fundamentals of Software Engineering, Rajib Mall , PHI, Latest edition
- 2) Software Engineering, I. Sommerville, Pearson Education, Asia.

Topics covered

- Introduction to Software Engineering
 - Software crisis
- Professional software development
 - Abstraction and decomposition
 - Software products and process
- Emergence of Software Engineering
- Software engineering ethics

Software Engineering

- An **engineering** approach to develop software
- Software engineering is an engineering discipline that is concerned with all aspects of software production from the early stages of system specification through to maintaining the system after it has gone into use.
- IEEE definition
 - *“The application of a **systematic, disciplined, quantifiable** approach to the development, operation and maintenance of software; that is, the application of engineering to software.”*

Software Engineering

- **Hardware vs. Software:** Almost all systems are software controlled – industrial manufacturing, national infrastructures, entertainment industry etc.
- Software systems are abstract and intangible.
- Software engineering is concerned with **theories, methods, tools and guidelines** for professional software development.

Software costs

- Software costs often dominate computer system costs. The costs of software on a PC are often greater than the hardware cost.
- Software costs more to maintain than it does to develop. For systems with a long life, maintenance costs may be several times development costs.
- Software engineering is concerned with cost-effective software development.

Software vs Hardware costs

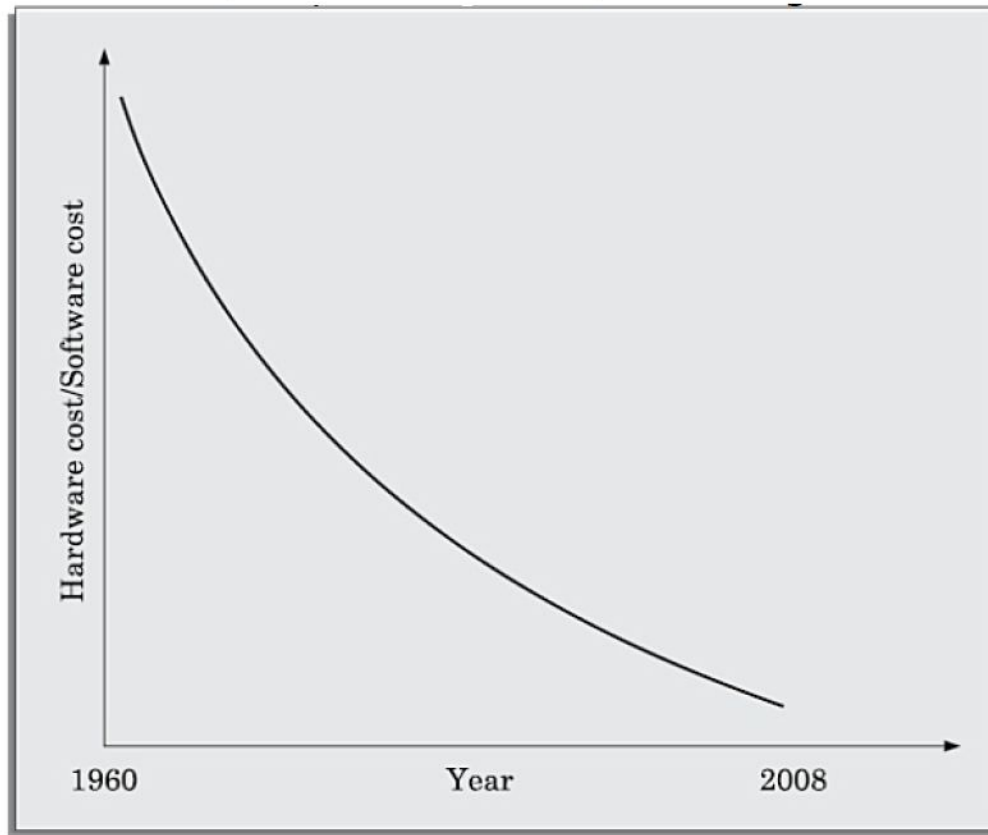


Figure1.1 Relative change in costs of hardware and software over time

Exploratory vs Professional software development

Exploratory Software Development

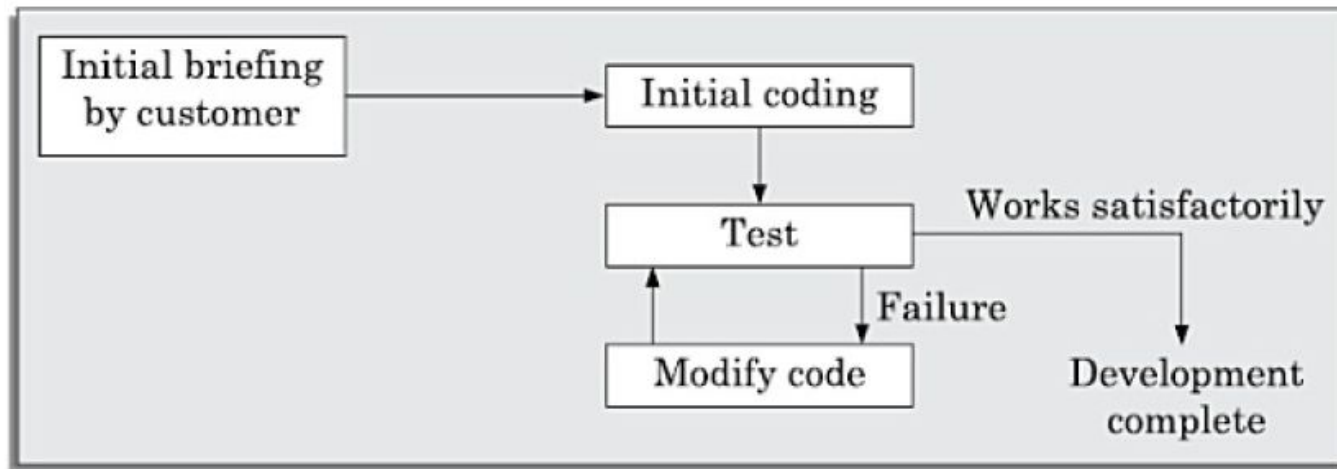


Figure1.2 Exploratory program development

Effect of Increasing Program size

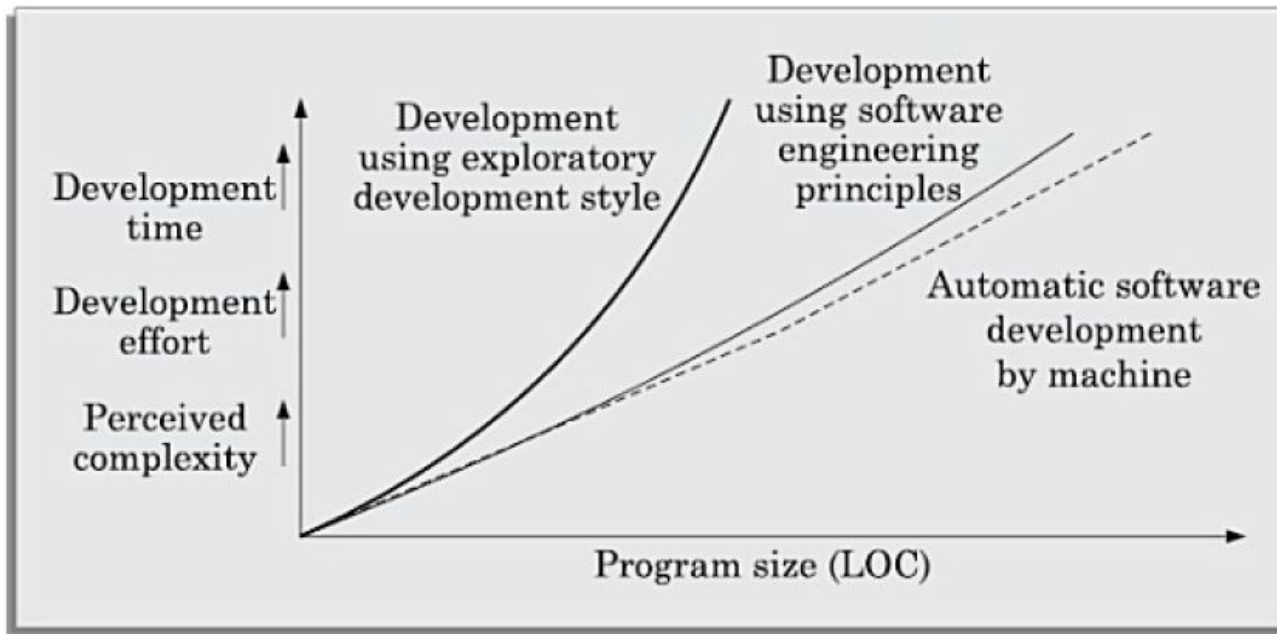


Figure1.3 Increase in development time and effort with problem size.

Software Crisis

- Fails to meet user requirements
- Expensive
- Unable to incorporate changes
- Exceeds delivery date
- Non-optimal usage of resources

Factors Contributing to Software Crisis

- Larger problems – increasing system complexity
- Poor project management
- Lack of adequate training in software engineering
- Increasing skill shortage
- Low productivity improvements

Professional software development

Fundamental Principle - Abstraction

- Hiding complexities of the problem under one abstract aspect
- Several/hierarchical abstractions of the same problem can be created
- Example – Using map to represent a country
- **Model construction**

Fundamental Principle - Decomposition

- Decompose a problem into smaller problems
- An arbitrary decomposition may result in difficulty to solve the problem
- Example - Organize a book into independent chapters rather than mixed topics

Software products

Generic products -

- Stand-alone systems that are marketed and sold to any customer who wishes to buy them.
- Examples – PC software such as graphics programs, project management tools; software for specific markets such as appointments systems for dentists.
- The specification of what the software should do is owned by the software developer and decisions on software change are made by the developer.
- **Horizontal** and **Vertical** market software

Software products

Customized products -

- Software that is commissioned by a specific customer to meet their own needs.
- Examples – air traffic control software, transaction gateway for banking systems.
- The specification of what the software should do is owned by the customer for the software and they make decisions on software changes that are required.
- Also known as **Services***.

Essential attributes of good software

Product characteristic	Description
Maintainability	Software should be written in such a way so that it can evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.
Dependability and security	Software dependability includes a range of characteristics including reliability, security and safety. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.
Efficiency	Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilisation, etc.
Acceptability	Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable and compatible with other systems that they use.

Software process

Software Process

- **Software specification**, where customers and engineers define the software that is to be produced and the constraints on its operation.
- **Software development**, where the software is designed and programmed.
- **Software validation**, where the software is checked to ensure that it is what the customer requires.
- **Software evolution**, where the software is modified to reflect changing customer and market requirements.

Software Process Basics

- Systems should be developed using a managed and understood development process. Of course, different processes are used for different types of software.
- Understanding and managing the software specification and requirements are important.
- Dependability and performance are important for all types of system. Thus, validation step should be as rigorous as possible.
- Where appropriate, you should reuse software that has already been developed rather than write new software.

General issues that affect software process

- Heterogeneity
- Business and social change
- Security and trust
- Scale of the software

Software engineering diversity

Types of Applications

- Stand-alone applications
 - These are application systems that run on a local computer, such as a PC. They include all necessary functionality and do not need to be connected to a network.
- Interactive transaction-based applications
 - Applications that execute on a remote computer and are accessed by users from their own PCs or terminals. These include web applications such as e-commerce applications.
- Embedded control systems
 - These are software control systems that control and manage hardware devices. Numerically, there are probably more embedded systems than any other type of system.

Types of Applications

- Batch processing systems
 - These are business systems that are designed to process data in large batches. They process large numbers of individual inputs to create corresponding outputs.
- Entertainment systems
 - These are systems that are primarily for personal use and which are intended to entertain the user.
- Systems for modeling and simulation
 - These are systems that are developed by scientists and engineers to model physical processes or situations, which include many separate interacting objects.

Types of Applications

- Data collection systems
 - These are systems that collect data from their environment using a set of sensors and send that data to other systems for processing.
- Systems of systems
 - These are systems that are composed of a number of other software systems.
 - Operational independence of components
 - Managerial independence of components.

Frequently asked questions about software engineering

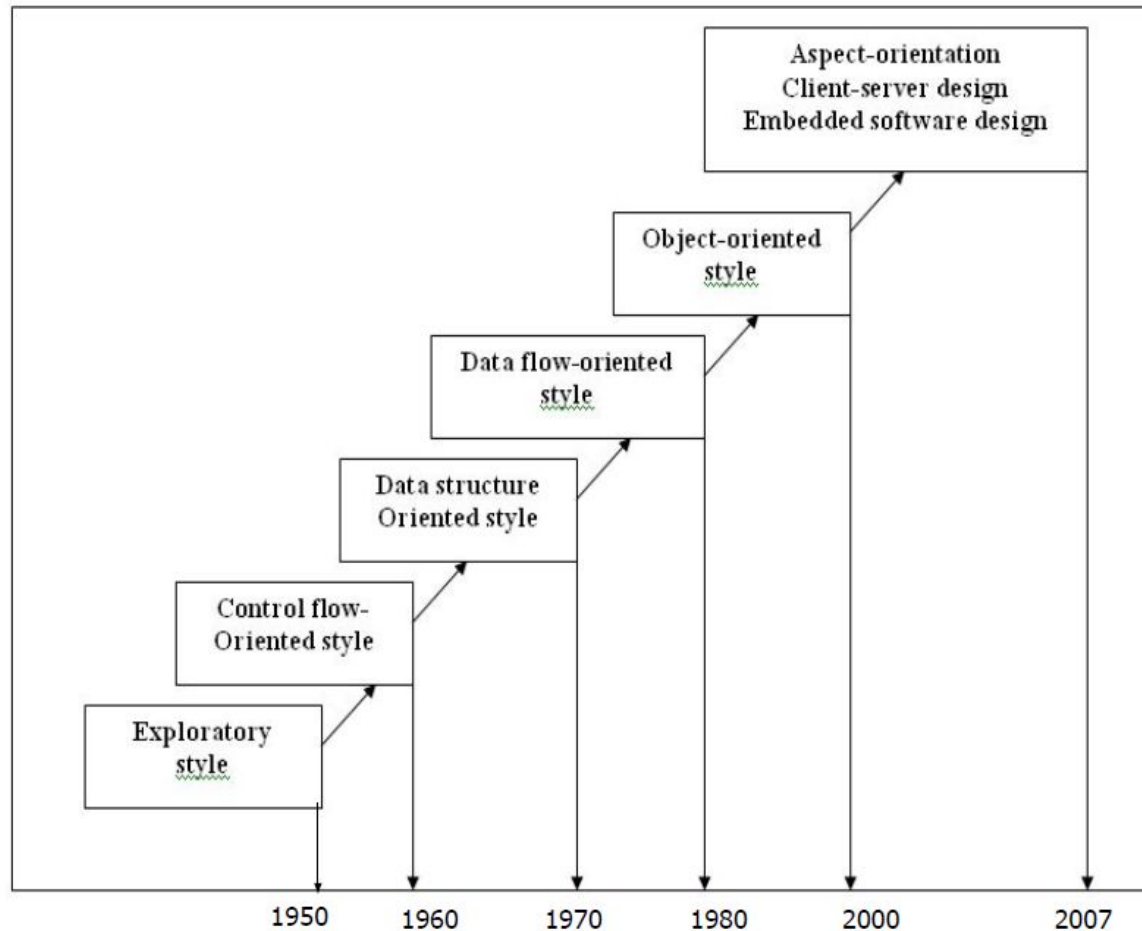
Question	Answer
What is software?	Computer programs and associated documentation. Software products may be developed for a particular customer or may be developed for a general market.
What are the attributes of good software?	Good software should deliver the required functionality and performance to the user and should be maintainable, dependable and usable.
What are the fundamental software engineering activities?	Software specification, software development, software validation and software evolution.
What is the difference between software engineering and computer science?	Computer science focuses on theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software.
What is the difference between software engineering and system engineering?	System engineering is concerned with all aspects of computer-based systems development including hardware, software and process engineering. Software engineering is part of this more general process.

Frequently asked questions about software engineering

Question	Answer
What are the key challenges facing software engineering?	Coping with increasing diversity, demands for reduced delivery times and developing trustworthy software.
What are the costs of software engineering?	Roughly 60% of software costs are development costs, 40% are testing costs. For custom software, evolution costs often exceed development costs.
What are the best software engineering techniques and methods?	While all software projects have to be professionally managed and developed, different techniques are appropriate for different types of system. For example, games should always be developed using a series of prototypes whereas safety critical control systems require a complete and analyzable specification to be developed. You can't, therefore, say that one method is better than another.
What differences has the web made to software engineering?	The web has led to the availability of software services and the possibility of developing highly distributed service-based systems. Web-based systems development has led to important advances in programming languages and software reuse.

Emergence of Software Engineering

Emergence of Software Engineering



Exploratory Style

- 1950s –
 - Programmers developed program based on their intuition using Assembly language programming
- 1960s (early) –
 - High level languages such as FORTRAN, ALGOL, and COBOL were introduced
 - Reduced the effort greatly
 - But still the development was done mostly in exploratory style

Control Flow Design

- 1960s (late) –
 - Program's control structure is the sequence in which the control of the program flow or the instructions are executed
 - Flowcharting techniques were developed
- Issues with Control Flow Design
 - Messy flow chart leads to difficulty in understanding and debugging
 - Go To statements make the flow of control alter arbitrarily
 - Introduction of structured programming languages

Data Structure Oriented Design

- 1970s (Early) –
 - Programmers focused on the data structures more than the control flow of the program
 - JSP* technique was developed
 - The data structure design is used to design/derive the program structure

Data Flow Oriented Design

- 1970s (Late) –
 - The data items input to a system has to be identified first
 - The processes that are required on the data items to produce the outputs should be determined.
 - Data Flow Diagram (DFD) design – functions/stations are identified

Object Oriented Design

- 1980s –
 - Identifies the objects involved in the problem first
 - Derive the relationship between different objects
 - Inheritance, composition, association etc.
 - Helps in data abstraction

Software Engineering Ethics

Software engineering ethics

- Software engineering involves wider responsibilities than simply the application of technical skills.
- Software engineers must behave in an honest and ethically responsible way if they are to be respected as professionals.
- Ethical behaviour is more than simply upholding the law but involves following a set of principles that are morally correct.

Issues of professional responsibility

- Confidentiality
 - Engineers should normally respect the confidentiality of their employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.
- Competence
 - Engineers should not misrepresent their level of competence. They should not knowingly accept work which is outwith their competence.

Issues of professional responsibility

- Intellectual property rights
 - Engineers should be aware of local laws governing the use of intellectual property such as patents, copyright, etc. They should be careful to ensure that the intellectual property of employers and clients is protected.
- Computer misuse
 - Software engineers should not use their technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine, say) to extremely serious (dissemination of viruses).

ACM/IEEE Code of Ethics

- The professional societies in the US have cooperated to produce a code of ethical practice.
- Members of these organisations sign up to the code of practice when they join.
- The Code contains eight Principles related to the behaviour of and decisions made by professional software engineers, including practitioners, educators, managers, supervisors and policy makers, as well as trainees and students of the profession.

Rationale for the code of ethics

- *Computers have a central and growing role in commerce, industry, government, medicine, education, entertainment and society at large. Software engineers are those who contribute by direct participation or by teaching, to the analysis, specification, design, development, certification, maintenance and testing of software systems.*
- *Because of their roles in developing software systems, software engineers have significant opportunities to do good or cause harm, to enable others to do good or cause harm, or to influence others to do good or cause harm. To ensure, as much as possible, that their efforts will be used for good, software engineers must commit themselves to making software engineering a beneficial and respected profession.*

The ACM/IEEE Code of Ethics

Software Engineering Code of Ethics and Professional Practice

ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices

PREAMBLE

The short version of the code summarizes aspirations at a high level of the abstraction; the clauses that are included in the full version give examples and details of how these aspirations change the way we act as software engineering professionals. Without the aspirations, the details can become legalistic and tedious; without the details, the aspirations can become high sounding but empty; together, the aspirations and the details form a cohesive code.

Software engineers shall commit themselves to making the analysis, specification, design, development, testing and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the following Eight Principles:

Ethical principles

1. PUBLIC - Software engineers shall act consistently with the public interest.
2. CLIENT AND EMPLOYER - Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.
3. PRODUCT - Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.
4. JUDGMENT - Software engineers shall maintain integrity and independence in their professional judgment.
5. MANAGEMENT - Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.
6. PROFESSION - Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.
7. COLLEAGUES - Software engineers shall be fair to and supportive of their colleagues.
8. SELF - Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

Ethical dilemmas

- Disagreement in principle with the policies of senior management.
- Your employer acts in an unethical way and releases a safety-critical system without finishing the testing of the system.
- Participation in the development of military weapons systems or nuclear systems.

Let's review!

Next Session

- Software Process Models